

结题验收

1 结题验收 5 分钟现场演示讲稿

日期：2026-05-20

适用场景：结题验收现场演示。本文按 5 分钟设计，精确到每一步打开哪个页面、做什么操作、说什么说明。

讲解原则：现场不讲函数级细节，不展开具体代码片段；只讲“页面背后的模块职责”和“CEBRA-WP 控制流如何落到工程结构中”。

1.1 0. 上台前准备

上台前先启动服务，不把启动时间算入 5 分钟演示。

```
PROTEIN_ENABLE_DEMO_FIXTURES=1 \  
UV_CACHE_DIR=/tmp/uv-cache \  
uv run python scripts/run_demo.py --port 8000 --no-smoke-test
```

另开终端注入答辩演示数据：

```
curl -X POST http://127.0.0.1:8000/demo/defense-full-flow
```

浏览器提前打开 5 个标签页：

```
http://127.0.0.1:8000/ui  
http://127.0.0.1:8000/ui/task-builder  
http://127.0.0.1:8000/ui/tasks/demo_defense_hitl  
http://127.0.0.1:8000/ui/tasks/demo_defense_done  
http://127.0.0.1:8000/ui/tasks/demo_defense_done/events
```

1.2 1. 0:00 - 0:35 开场：系统定位

操作：打开 /ui Dashboard。

讲稿：

各位老师好，我现在演示的是论文中的蛋白质设计多 Agent 工作流系统。这个系统不是单次调用一个大模型，而是把蛋白质设计任务组织成可规划、可执行、可恢复、可审计的工作流。

论文第四章中，我把系统分成任务接入、规划、执行、安全审查、结果汇总和工具资源层。这里 Dashboard 展示的是任务入口和任务状态，后续我会沿着一条固定演示链路展示输入、HITL 决策、执行结果、结构查看和事件审计。

现场演示使用本地确定性 demo fixture，不调用远程 LLM 或 OpenFold3 服务，目的是稳定展示系统机制。真实实验结果在论文第七章和冻结实验产物中说明。

代码层补充，可视情况插入：

从代码结构上看，后端入口主要在 src/api，数据契约在 src/models，控制流在 src/workflow，多 Agent 职责在 src/agents。前端页面只负责展示和提交人工决策，不直接改变工作流状态。这样可以保证算法控制流集中在后端，而不是分散在 UI 里。

1.3 2. 0:35 - 1:15 输入解析与任务录入

操作：切到 /ui/task-builder。

讲稿：

这里是任务录入页面。系统入口支持自然语言任务描述，并进一步收敛到结构化任务字段。

本次演示任务是：评估一个 TRP-cage-like 短肽序列的稳定性，优先采用低成本结构预测路径，并在高代价步骤前展示候选方案给人工确认。

如果页面上有字段，指着字段继续说：

可以看到任务会被整理为 sequence、objective type、budget policy、runtime policy、是否需要人工审查等字段。这样做的原因是，后续 PlannerAgent 不能只根据一句自然语言直接执行，而是要基于明确约束生成候选计划。

这对应论文第三章的任务接入需求，以及第四章中“目标、约束、工具能力和预算共同决定候选工作流”的设计。

代码层补充，可视情况插入：

代码里这部分对应任务接入和数据模型层。前端 Task Builder 收集输入，后端 API 把它转成统一的任务对象和约束字段。后续 Planner、Executor、RuntimeEvaluator 都不直接依赖页面表单，而是依赖这些结构化契约。

1.4 3. 1:15 - 2:45 HITL 候选比较与算法解释

操作：切到 `/ui/tasks/demo_defense_hitl`。先停留在 `pending action / candidate comparison` 区域，再依次指向候选评分、运行状态 JSON 和右侧“理论对象”面板。

讲稿：

现在这个任务处于 `WAITING_PATCH_CONFIRM` 状态。这个状态是有限状态机中的等待态，含义是系统已经暂停执行，正在等待人工确认。

这里展示的是一个高代价步骤前的 `patch` 确认场景：远程结构预测服务 `readiness` 降级，系统没有继续盲目执行，而是生成了三个候选方案。

指向三个候选，说：

- 第一个 `patch_local_openfold` 是默认推荐，风险低、成本低，分数最高。
- 第二个 `suffix_replan_low_cost` 是保留已完成前缀，改用低成本后缀重规划。
- 第三个 `patch_remote_retry` 是继续重试远程服务，但成本和风险更高。

这里的重点不是让 AI 自动替我批准，而是系统给出候选、分数、风险、成本和解释，由人来做最终决策。

代码层补充，可视情况插入：

这一页背后的关键代码不是前端按钮，而是后端的状态机和 `PendingAction` 契约。系统进入 `WAITING_PATCH_CONFIRM` 之前，会把当前任务、候选 `patch`、运行时状态摘要和事件日志写好；之后前端只是读取这些对象并展示给人。也就是说，人工确认是控制流的一部分，而不是临时弹窗。

1.4.1 3.0 解释候选方案中的决策证据

操作：在候选方案卡片中指向“决策证据”“`metadata`”“`tool readiness`”或类似区域。如果页面需要展开候选详情，先展开默认推荐 `patch_local_openfold`。

默认推荐候选可以按下面这组证据解释：

```
{  
  "candidate_id": "patch_local_openfold",  
  "risk_level": "low",
```

```

"cost_estimate": "low",
"affected_steps": ["S2"],
"expected_effect": "Preserve S1 and patch S2 before resuming.",
"recovery_semantics": "patch_local",
"tool_id": "openfold",
"capability_id": "structure_prediction",
"io_type": "sequence_to_structure",
"adapter_mode": "local",
"tool_readiness": {
  "status": "ready",
  "reason": "Local deterministic fixture is available."
}
}

```

讲稿：

这里每个候选方案都有一组决策证据。candidate_id 表示候选动作的唯一标识；risk_level 和 cost_estimate 是给人看的离散风险和成本等级，便于现场快速比较。

affected_steps 表示这个 patch 只影响 S2，也就是结构预测步骤；expected_effect 说明它会保留 S1 的成功前缀，只替换后续高代价结构预测路径。这一点对应 CEBRA-WP 中的恢复优先级：能局部修补时，优先 patch，而不是直接全量 replan。

recovery_semantics 是 patch_local，说明这是一个局部工具替换型恢复动作。tool_id 是 openfold，capability_id 是 structure_prediction，io_type 是 sequence_to_structure，表示这个候选仍然满足“输入序列、输出结构”的工具契约。

adapter_mode 是 local，tool_readiness 是 ready，说明这个候选当前可用；相比之下，远程 retry 候选虽然保持原计划，但 readiness 降级、成本更高，所以不会成为默认推荐。

再指向另外两个候选，补充：

suffix_replan_low_cost 的语义是保留前缀、替换后缀路径，它比局部 patch 改动更大，所以成本和扰动通常高一些。

patch_remote_retry 的目标匹配可能不低，因为它保留了原来的远程 OpenFold3 路径；但它的 adapter_mode 是 remote，readiness 是 degraded，所以 risk 和 cost 分数被拉低。

因此候选排序不是只看目标匹配，而是同时看工具契约是否闭合、当前工具是否 ready、会影响哪些步骤、是否保留成功前缀、风险成本是否可接受。这就是论文里“约束感知、预算感知、恢复自适应”的含义。

代码层补充，可视情况插入：

在代码层，这些字段对应候选对象的统一数据契约。Planner 负责产生候选和解释，workflow 层负责判断候选是否进入等待态，models 层保证 candidate_id、risk、cost、payload 和 metadata 都是结构化字段。这样后续审计时不会只剩一段自然语言理由。

1.4.2 3.1 解释候选评分 JSON

操作：指向候选卡片中的 score breakdown。如果页面展示 JSON 或可展开详情，就展开评分字段。

页面中的评分 JSON：

```
{  
  "feasibility": 0.88,  
  "objective": 0.82,  
  "risk": 0.9,  
  "cost": 0.86,  
  "overall": 0.86  
}
```

讲稿：

这里的评分对应论文第四章 CEBRA-WP 的静态效用部分。feasibility 表示这个候选在工具能力、输入输出契约和前后步骤衔接上是否可执行；objective 表示它和当前稳定性评估目标的匹配程度；risk 表示风险越低分越高；cost 表示成本越低分越高；overall 是综合分。

所以这个 0.86 不是大模型随口给出的分数，而是把可行性、目标匹配、风险和成本拆开之后再合成的候选效用。这样做的好处是，老师或者用户可以追问“为什么推荐它”，系统能够把依据拆开解释。

代码层补充，可视情况插入：

实现上，这类 score breakdown 由规划和候选评估模块统一生成。代码不会只保存 overall 一个数，而是保留 feasibility、objective、risk、cost 等分项。这样前端能展示“为什么这个候选排第一”，实验统计也能回溯每次排序依据。

1.4.3 3.1.1 这些候选分数是怎么算出来的

操作：仍停留在 score breakdown 区域。不要展开代码，只讲计算口径。

讲稿：

这里需要说明一下这些数值的来源。现场 demo 为了稳定，候选分数是 fixture 中固定好的代表值；但真实系统里不是手填一个 overall，而是由候选工具链的工具元数据、风险、成本、readiness 和恢复性计算

出来。

以 feasibility 为例，代码里会看工具覆盖度和 fallback 深度。直观上，候选使用的工具越能覆盖当前任务能力、替代路径越充分，feasibility 越高。

risk 和 cost 是反向分数：工具风险越低，risk score 越高；工具成本越低，cost score 越高。因此这里 risk 为 0.90、cost 为 0.86，表示 local patch 这条路径相对低风险、低成本。

objective 表示候选和任务目标的匹配程度。这个 demo 的目标是稳定性评估，所以 local OpenFold 兼容路径可以产出结构证据，能够继续支持后续稳定性代理评分。

overall 是把这些分项按权重合成后的候选基础分。在论文公式里，对应静态效用 S_{static} ；如果后续已有目标证据，则 objective 项会被后验目标评分替换，形成 S_{post} 。

可以补一句实现映射：

```text

从代码结构看，Planner 或候选评估模块负责生成 score\_breakdown；RuntimeEvaluator 不重新判断工

#### 1.4.4 3.2 解释运行状态 JSON

操作：指向运行状态 / runtime state 区域。如果页面展示 JSON，就停留 5 到 8 秒。

页面中的运行状态 JSON：

```
{
 "schema_version": 1,
 "p_success": 0.64,
 "p_structural_failure": 0.31,
 "recovery_margin": 0.72,
 "expected_remaining_cost": 1.35,
 "evidence_sufficiency": 0.58,
 "budget_pressure": 1.12,
 "budget_cap": 1.2
}
```

讲稿：

这里是 Lite belief-state，也就是论文里说的轻量信念状态。它不是完整 POMDP，也不是强化学习控制器，而是一个可解释的运行状态向量。

p\_success 为 0.64，表示在当前观测下继续恢复执行的成功倾向是中等偏高；p\_structural\_failure 为 0.31，说明结构相关失败风险并不为零，所以不能直接忽略。

recovery\_margin 为 0.72，表示已完成前缀还有保留价值，适合优先做局部 patch，而不是直接全量重规划。expected\_remaining\_cost 为 1.35，说明后面还有一定成本暴露。

evidence\_sufficiency 为 0.58，说明证据不是完全充分；budget\_pressure 为 1.12，接近 budget\_cap 1.2，说明预算压力已经比较明显。因此系统选择进入 WAITING\_PATCH\_CONFIRM，让人确认是否采用低成本 patch。

算法收束：

从算法角度看，这一步就是 CEBRA-WP 的运行时自适应层：系统先根据 StepResult、tool readiness 和历史执行结果更新 runtime state，再用这个状态解释为什么此时不直接继续高代价远程调用，而是生成 patch\_confirm 的 PendingAction。

代码层补充，可视情况插入：

代码里 runtime state 的更新集中在 workflow 运行时层，而不是由某个 Agent 自己随意改状态。Executor 产生 StepResult，SafetyAgent 产生风险判断，workflow 层把这些观测统一汇入 RuntimeState。这样做是为了保证 CEBRA-WP 的状态更新可复现、可测试、可写入快照。

### 1.4.5 3.2.1 这些运行时状态数值是怎么算出来的

操作：指向 runtime state JSON，尤其是 expected\_remaining\_cost、budget\_pressure、evidence\_sufficiency。

讲稿：

这里的 runtime state 在 demo 中同样是固定 fixture，用于稳定演示等待态。但真实系统里它来自执行观测的规则化更新。

初始状态是一个保守的中性估计，例如 p\_success 从 0.5 附近开始，p\_structural\_failure 从 0.25 附近开始，recovery\_margin 从 0.6 附近开始。执行过程中，如果一个步骤成功，p\_success 会增加，p\_structural\_failure 会下降，recovery\_margin 会增加；如果结构相关步骤失败，p\_structural\_failure 会明显上升，recovery\_margin 会下降。

evidence\_sufficiency 不是一次性跳变，而是平滑更新。它由廉价验证覆盖、候选一致性、指标完整性三类信号汇总，再和上一轮 evidence\_sufficiency 做平滑，避免一次偶然观测让系统过度反应。

expected\_remaining\_cost 表示从当前状态继续到结束还暴露多少成本。budget\_pressure 则是从 expected\_remaining\_cost 和预算上限派生出来的。这里 expected\_remaining\_cost 是 1.35，budget\_cap

是 1.2，所以 `budget_pressure` 大约是  $1.35 / 1.2$ ，也就是 1.12。这个数超过 1，说明已经接近或略高于预算压力线。

再把它和决策连起来：

所以这里系统不是因为单个阈值触发等待，而是同时看到：成功概率还可以、结构失败风险不低、恢复余量还够、证据不完全充分、预算压力偏高。这个组合更适合让人确认一个低成本 local patch，而不是直接继续远程高代价调用。

### 1.4.6 3.3 解释右侧“理论对象”面板

操作：指向右侧“理论对象”面板，按从上到下的顺序解释。

讲稿：

右侧这个“理论对象”面板是把论文里的算法变量映射到页面上。

静态分 0.86 对应候选在运行时观测介入前的先验效用，也就是由 `feasibility`、`objective`、`risk` 和 `cost` 得到的分数。

运行时调整这里显示为横线，表示这个 demo 卡片没有额外展示单独的 `delta` 标量；运行时状态单独展示在前面的 JSON 中。最终分 0.86 表示当前用于排序和推荐的候选效用。

选中操作是补丁确认，说明算法不是直接执行 patch，而是把 patch 作为候选动作提交给 HITL。证据充分性 0.58 和预算压力 1.12 分别对应 Lite belief-state 中的 `evidence_sufficiency` 和 `budget_pressure`，它们共同解释为什么这里需要人工确认。

再用一句话靠近公式：

如果用论文里的形式表达，这里就是先计算候选的静态效用  $S_{\text{static}}(\pi)$ ，再在有目标证据时形成  $S_{\text{post}}(\pi)$ ，最后叠加运行时修正  $\Delta(\pi, x_t)$ ，得到运行时效用  $U(\pi, x_t)$ 。这里展示的 demo 没有额外显示 `delta` 标量，所以最终分和静态分相同；但控制流仍然体现了 CEBRA-WP 的思想：算法给出推荐动作，FSM 和 HITL 决定是否推进。

代码层补充，可视情况插入：

代码中对应的是 `RuntimeEvaluator` 这一类运行时评估组件。它不负责调用模型，也不负责改 UI，而是把候选分数、运行时状态和动作偏好合成为可展示的 `final score`、`runtime adjustment` 和 `action utility`。换句话说，它是 CEBRA-WP 在工程里的主要算法落点之一。

### 1.4.7 3.4 如果老师追问：动作效用怎么影响 patch / replan / stop

操作：仍停留在 HITL 页面，指向“选中操作：补丁确认”或 action utility / 理论对象区域。

讲稿：

CEBRA-WP 不只给候选排序，还会估计四类控制动作的效用：continue、patch\_local、suffix\_replan 和 stop。

continue 偏好高 p\_success、高 evidence\_sufficiency、低 structural failure 和低 budget pressure。patch\_local 偏好较高 recovery\_margin、局部可修复性和可复用证据。suffix\_replan 偏好结构性失败压力较高、当前成功率下降、但前缀仍可保留的场景。stop 只有在成功率很低、预算压力高、恢复余量低，并且人工介入价值也低时才会成为强候选。

以这个 demo 的数值看，p\_success 是 0.64，recovery\_margin 是 0.72，说明当前还没到 stop；p\_structural\_failure 是 0.31，budget\_pressure 是 1.12，又说明继续远程高代价调用不划算。因此系统把动作落到 patch\_confirm，让人确认局部修补路径。

代码层补充，可视情况插入：

实现上，动作效用由 RuntimeEvaluator 根据 RuntimeState 和派生特征计算；但最终是否进入 WAITING\_PATCH\_CONFIRM、是否应用 Decision，仍由 workflow/FSM 控制。也就是说，算法提供效用和建议，系统控制流负责合法迁移。

## 1.5 4. 2:45 - 3:25 完成态结果与报告

操作：切到 /ui/tasks/demo\_defense\_done。先展示状态，再停留在报告浏览器卡片，依次指向 sequence、scores、objective scoring、top-k、warnings、evidence refs 和 structure similarity。

讲稿：

这里展示的是同一条链路在人工接受推荐 patch 后的完成态。任务状态已经进入 DONE。

结果中包含序列、结构文件、评分、风险标记和报告路径。这里的报告不是单纯一段自然语言总结，而是把 objective scoring、structure similarity、证据来源和 warning 一起保存下来。

### 1.5.1 4.1 解释报告浏览器中的基础字段

操作：指向报告浏览器中的任务来源、序列、结构路径和 scores 区域。

页面中的报告核心字段：

```
{
 "task_id": "demo_defense_done",
 "source": "defense_demo_fixture",
 "sequence": "NLYIQWLKDGGPSSGRPPPS",
 "structure_pdb_path": "output/demo/defense-full-flow/demo_defense_done.pdb",
 "scores": {
 "plddt_mean": 88.2,
 "stability_proxy": 0.81,
 "sequence_length": 20,
 "qc_pass": true
 }
}
```

讲稿：

报告浏览器首先展示任务结果的基础字段。task\_id 用来把报告和任务、事件日志、结构文件关联起来；source 标记这个结果来自 defense\_demo\_fixture，说明这是本地确定性演示数据。

sequence 是本次评估的短肽序列；structure\_pdb\_path 是结构查看器实际加载的 PDB 文件路径。也就是说，前端看到的三维结构不是孤立页面，而是 DesignResult 报告中的一个可追溯产物。

scores 这一组是结果层指标。plddt\_mean 表示结构预测置信度的演示指标；stability\_proxy 是稳定性的代理分数；sequence\_length 是序列长度；qc\_pass 表示质量门禁是否通过。

这些字段来自执行完成后的 DesignResult 和报告产物。在真实链路里，它们会由结构预测工具、质量检查工具和 SummarizerAgent 汇总；在现场 demo 中由 fixture 固定生成，用来稳定展示报告契约和前端渲染。

代码层补充，可视情况插入：

代码上，报告不是前端临时拼出来的，而是后端结果契约的一部分。工具适配层产生结构和指标，workflow 汇总执行状态，SummarizerAgent 负责把已有证据整理成 DesignResult。前端报告浏览器只是读取这个结构化结果。

### 1.5.2 4.2 解释 objective scoring

操作：指向 objective scoring 卡片。

页面中的 objective scoring：

```
{
```

```

"objective_score": 0.84,
"posterior_score": {
 "aggregate_score": 0.84,
 "evidence_status": "sufficient_for_demo"
}
}

```

讲稿：

objective\_score 是结果相对任务目标的综合目标分。在这个 demo 中，目标是稳定性评估，所以它把结构置信、稳定性代理分和质量门禁这些结果证据汇总成一个面向目标的分数。

posterior\_score 表示后验评分。这里的“后验”是相对于执行前候选评分而言的：候选阶段只有工具能力、风险、成本和 runtime state；完成后系统已经有结构文件、质量指标和事件证据，所以可以形成结果层的 aggregate\_score。

evidence\_status 是 sufficient\_for\_demo，表示这些证据足够支撑演示中的报告展示，但不等价于真实湿实验验证。这和论文结论边界一致。

算法衔接：

从 CEBRA-WP 角度看，前面的 HITL 页面展示的是候选动作的运行时效用  $U(\pi, x_t)$ ，这里展示的是执行完成后的目标证据和后验结果评分。两者共同构成“先决策、后验证、可追溯”的闭环。

代码层补充，可视情况插入：

实现中，这一层对应 posterior objective scoring 的结果绑定。也就是候选阶段先有  $S_{static}$ ，执行后如果产生了结构、质量和目标证据，就把 objective 部分替换为后验目标分，形成  $S_{post}$ 。最终 runtime adjustment 叠加在这个基础分上，而不是直接覆盖掉原来的候选可行性判断。

### 1.5.3 4.3 解释 top-k 候选后验结果

操作：指向 top-k / candidate ranking 区域。

页面中的 top-k：

```

[
 {
 "candidate_id": "patch_local_openfold",
 "objective_score": 0.86,
 "posterior_score": {
 "evidence_status": "supported"
 }
 }
]

```

```

 }
 },
 {
 "candidate_id": "suffix_replan_low_cost",
 "objective_score": 0.78,
 "posterior_score": {
 "evidence_status": "supported"
 }
 },
 {
 "candidate_id": "patch_remote_retry",
 "objective_score": 0.71,
 "posterior_score": {
 "evidence_status": "degraded_remote_readiness"
 }
 }
]

```

讲稿：

这里的 top-k 不是重新执行三条路径，而是把候选阶段的方案和完成后的证据放在同一个报告里，便于复盘为什么默认推荐 patch\_local\_openfold。

patch\_local\_openfold 的 objective\_score 为 0.86，并且 evidence\_status 是 supported，说明它既是候选阶段的默认推荐，也能被完成态证据支持。

suffix\_replan\_low\_cost 也被标记为 supported，但分数较低，说明它作为备选恢复路径是合理的，只是改动范围和成本更大。

patch\_remote\_retry 的 evidence\_status 是 degraded\_remote\_readiness，说明它的问题不是目标不相关，而是运行时工具 readiness 降级导致风险和成本不可接受。

这体现了 CEBRA-WP 的一个关键点：候选排序不是只看一个最终分，而是把候选来源、工具状态、成本风险、运行时证据和完成态证据一起保留下来，支持事后审计。

代码层补充，可视情况插入：

代码里保留 top-k 的原因，是为了让系统不是单轨迹黑箱执行。即使最终选择了默认候选，其他候选的分数和证据状态也会保留，用于 HITL 比较、失败复盘和实验统计。这也是它和普通“LLM 生成一条计划然后执行”的主要区别。

### 1.5.4 4.4 解释 warnings、evidence refs 和 structure similarity

操作：指向 warnings、evidence refs、structure similarity 区域。

页面中的字段：

```
{
 "warnings": [
 "demo fixture; no remote inference was executed"
],
 "evidence_refs": [
 {
 "type": "event_log",
 "path": "data/logs/demo_defense_done.jsonl"
 },
 {
 "type": "pdb",
 "path": "output/demo/defense-full-flow/demo_defense_done.pdb"
 }
],
 "structure_similarity": {
 "hit_count": 3,
 "top_hit": {
 "hit_id": "TRP_CAGE_REFERENCE",
 "tm_score": 0.73,
 "rmsd": 2.1
 }
 }
}
```

讲稿：

warnings 用来明确报告边界。这里写明没有执行远程推理，所以我不会把这个结构解释成真实模型实时预测结果。

evidence\_refs 是证据索引。event\_log 指向事件日志，可以追溯任务状态、等待态、人工决策和完成过程；pdb 指向结构文件，可以被结构查看器加载。

structure\_similarity 是结构相似性摘要。hit\_count 表示找到 3 个参考匹配；top\_hit 是 TRP\_CAGE\_REFERENCE；tm\_score 和 rmsd 是结构相似性指标。这里它们用于演示报告

字段和结构证据组织方式，不用于宣称真实生物学功能。

所以报告浏览器的作用有三个：第一，把结果从“页面展示”变成可保存的数据契约；第二，把算法决策、结构产物和事件日志连起来；第三，为老师追问“这个结论从哪里来”提供可回溯证据。

指向 warning 或 source 字段，说：

可以看到 demo artifact 明确标记为本地演示产物，不把它包装成真实远程模型推理结果。这一点和论文中的边界一致：本文验证的是 workflow 机制、恢复控制和审计能力，不宣称通过现场演示证明蛋白质真实生物学功能。

## 1.6 5. 3:25 - 3:55 三维结构查看

操作：在同一个任务详情页中找到 Structure Viewer。旋转、缩放一次，点击或悬停结构。

讲稿：

这里是结构查看器，系统可以直接加载任务产出的 PDB 文件，并在前端展示三维结构。

在真实使用中，这个位置可以接入 ESMFold、OpenFold3 或其他结构预测工具的输出。现场演示使用固定 PDB，是为了保证答辩时页面稳定。系统关心的是结构产物能否被记录、展示、评分和追溯。

解释模块划分：

从模块上看，结构文件由工具适配层或 demo fixture 产生，后端 API 提供结构接口，前端 Task Detail 负责展示；工作流状态仍由 src/workflow 维护，不由前端直接改状态。

## 1.7 6. 3:55 - 4:35 事件时间线与审计

操作：切到 /ui/tasks/demo\_defense\_done/events。

讲稿：

最后看事件时间线。这里记录了从任务创建、规划、运行、工具 readiness 降级、runtime state 更新、创建 pending action、进入等待态、提交决策、应用决策、退出等待态，到最终总结和 DONE 的完整链路。

这对应论文中 EventLog 和 TaskSnapshot 的设计。它的价值是：系统不是黑箱地给出一个结果，而是可以回放“为什么停下来、为什么推荐这个候选、谁做了决策、之后状态如何变化”。

指向 WAITING\_ENTER、DECISION\_APPLIED、WAITING\_EXIT、DONE：

这几个事件体现了 HITL 的关键边界：进入等待态后执行暂停，人工决策写入后才继续推进，最后进入完成态。

代码层补充，可视情况插入：

代码里 EventLog 和 TaskSnapshot 是控制流审计的核心。Workflow 层每次进入 WAITING、应用 Decision、退出等待态、进入 DONE，都会留下事件记录。这样 CEBRA-WP 的动作建议不是只在页面上显示一下，而是和状态迁移、人工决策、产物路径一起固化下来。

## 1.8 7. 4:35 - 5:00 实验结果与收束

操作：回到 Dashboard 或停留在事件页都可以，不再切复杂页面。

讲稿：

论文第七章中，我用 12 个 task key、4 组策略、共 84 次运行验证系统机制。最终 81 次 DONE、3 次 FAILED。

需要说明的是，static\_top1 在当前矩阵中成功率最高，为 100%；其他三组为 95.2%。所以我没有把结论写成“CEBRA-WP 显著提高成功率”。本文更稳妥的结论是：CEBRA-WP 机制链路可执行、Lite belief-state 可观测、恢复决策可审计，并且固定阈值门控会带来额外高代价调用。

总结来说，这个系统的核心贡献是把 LLM Agent、工具知识、FSM、HITL、运行时恢复和审计日志组织成一个可复现的蛋白质设计 workflow 原型。

## 1.9 8. 老师打断时的短答

### 1.9.1 问：这是实时跑模型吗？

不是。现场为了稳定性使用本地确定性 fixture，不调用远程 LLM、PLM REST 或 OpenFold3 REST。它展示的是系统交互、FSM、HITL、结构查看和审计链路。真实实验结果来自论文第七章的 84-run 冻结实验矩阵。

### 1.9.2 问：为什么不用普通固定流水线？

普通固定流水线适合步骤稳定、成本均匀、失败语义简单的任务。但蛋白质设计 workflow 里，结构预测、精修和目标评分往往高成本，而且失败可能来自工具异常、输入质量不足或候选链路整体不可行。因此系统需要在运行时根据观测决定继续、patch、suffix replan 或 stop。

### 1.9.3 问：CEBRA-WP 的核心是什么？

它是工作流层的规划与恢复控制算法，不是新的蛋白质生成模型。核心过程是：候选生成、硬可行性过滤、静态评分、运行时状态更新、候选重排序和恢复动作选择。它的目标是在高代价、会失败的科研工作流中减少盲目执行，并保留可解释的恢复路径。

### 1.9.4 问：这个算法在代码中怎么落地？

我会从模块职责上回答。Planner 负责产生候选计划、patch 或 replan 候选；models 层定义 Plan、PendingAction、Decision、RuntimeState 和 DesignResult 这些数据契约；workflow 层负责 FSM、等待态、恢复和事件日志；RuntimeEvaluator 负责把候选分数、运行时状态和动作效用组合起来；前端只展示这些结构化对象并提交人工决策。这样 CEBRA-WP 是贯穿控制流的算法，而不是某一个孤立函数。

### 1.9.5 问：Lite belief-state 是什么？

它是一个轻量运行时状态向量，包含五个量：成功概率、结构性失败风险、恢复余量、剩余成本和证据充分性。它不是完整 POMDP 或强化学习控制器，而是为了让恢复决策可解释、可审计、可复现。

### 1.9.6 问：实验中 static\_top1 成功率更高，算法价值在哪里？

我没有把结论写成 CEBRA-WP 显著提高成功率。当前 84-run 矩阵支持的是机制结论：CEBRA-WP 链路可执行，Lite belief-state 可观测，HITL 和恢复决策可审计，固定阈值门控会带来额外高代价调用。算法价值主要在恢复控制、成本意识和审计解释，而不是单纯成功率提升。

### 1.9.7 问：系统缺陷是什么？

主要有四点。第一，84-run 实验规模有限，统计效力不足。第二，矩阵中真实 patch 主要出现在 fixed\_threshold\_gate 组，suffix\_replan 和 terminal\_stop 的矩阵级证据还不够。第三，系统仍是原型，数据库持久化、ToolKG 动态更新和远程服务故障切换还可以加强。第四，外部 Agent 基线还可以扩展，比如 ReAct、Tree of Thoughts 和 Reflexion。

## 1.10 9. 如果时间只剩 3 分钟

只讲这 4 个页面：

`http://127.0.0.1:8000/ui/task-builder`

`http://127.0.0.1:8000/ui/tasks/demo_defense_hitl`

`http://127.0.0.1:8000/ui/tasks/demo_defense_done`

`http://127.0.0.1:8000/ui/tasks/demo_defense_done/events`

压缩主线：

输入结构化 -> 高代价步骤前等待人工确认 -> 候选按风险成本和 runtime state 排序 -> 接受 patch 后完成 -> 报告、结构、事件日志可追溯。